

A's build sheet, Rain + agent side

Full reasoning lives in `BUILD-PLAN.md` and `docs/` in the repo. This page is just what you do, in order.

● First, before any code: two questions to a Rain engineer, together with B

1. **Can we simulate an authorization** against a card we issue? This decides whether the demo can show a decline, or only issuance.
2. **Does card issuance lock to one exact merchant, or only a category?** Decides whether the vendor-check point is “Rain would have allowed this” or the weaker “we stop it before Rain is asked.”
3. Base URL and auth header to use.
4. Is the collateral already funded, or do we send RUSD first.

Do this before 14:00. It shapes the demo, not just the build.

Your three-hour blocks

13:00-15:00: one authenticated call, nothing else matters yet

Goal: **get a single successful authenticated request to Rain working**. Not a card, not a flow, just proof the credentials and base URL work end to end.

- API key from the paper sheet into your own `.env.local`, never shared, never committed
- Team ID, User ID, Collateral contract ID are already in `.env.local.example`
- Hit whatever the lightest read endpoint is (contract details, credit balance) first
- Once that returns, you have everything you need to build on

15:00-17:00: issue a scoped virtual card, end to end

- `issueRainCard()` with a limit scoped to a single amount, short expiry
- Confirm it matches what a “scoped virtual card” actually looks like in Rain’s API response, card id, status, last four, expiration, limit with frequency and amount
- The agent side: a small task-taking agent that proposes a purchase. Keep it simple, its job is to produce the PO object B’s checker needs: `{ poNumber, vendor, sku, unitPrice, quantity, quoteExpiry }`
- **If there’s time before 17:00**, the quote stage: 2-3 seller agents give competing quotes, one counter-offer round, the winner becomes the PO. If this isn’t done by 17:00, skip it, one seller quote is fine, it’s the first thing that gets cut anyway.

● 17:00: the join

B's `verify()` function is ready. Wire it in: your agent's proposed PO goes through B's check first. Pass, you call `issueRainCard()`. Fail, you don't call Rain at all, you return B's refusal reason.

If this isn't working by 18:00, stop and cut scope. Don't push the join later hoping it comes together, cut the quote stage or simplify the agent instead.

18:00-20:00: the Monad anchor, and the run-it-twice path

- **Rule-version anchor on Monad testnet**, this is B's rule-versioning output, but you own the transaction: hash a rule version, send it, store the tx hash. One transaction per version, not per decision. This is the one that stays if time is short.
- Confirm the **run-it-twice** demo path actually works: issue a card for a PO, then submit the exact same PO again, confirm it gets refused by your idempotency check (B's rule 6) reading the record your first run wrote. This is the headline demo moment, test it for real, more than once.
- **Revoke the card, if the endpoint exists.** Ask about this alongside the other Rain questions. Deactivate after settlement. Confirmed by Rain's own framing, cards are "retired automatically once the job is done."

20:00-21:00: deploy, verify, rehearse

- Push, deploy
- **Verify the run-it-twice flow works on the deployed URL, from your phone**, not just localhost
- Rehearse your half of the pitch out loud, twice, timed

What you're explicitly not building

- No delegated budget trees, no Burn Rate streaming, no Four-Eyes voting. Reasoning in `THE-IDEA.md` if anyone asks why.
- No second Rain collateral identity. One team, one contract, the one on the paper sheet.
- Don't over-build the negotiation stage. Two or three seller agents, one round, done. If it's eating time, cut it, it's cut line #1 for a reason.

Handoff to B

The only thing B needs from you is the exact shape of what `issueRainCard()` returns and what a PO object looks like once your agent produces one. Agree that shape with B in the first thirty minutes, before either of you build much, so the 17:00 join isn't a surprise.